

Architecture Before Technology: the cheatsheet

Matthias Falland · FabCon Europe 2026 · Community Hub · [linkedin.com/in/matthias-falland](https://www.linkedin.com/in/matthias-falland) · Repo: github.com/TheTrustedAdvisor/fabcon-europe-2026

Decide the structure first; then the technology questions answer themselves. Answer the five questions before the first item is created, in this order: each answer makes the next one easier. Every answer below has the decision, one Fabric reason, the pitfall I see most, and a Monday check you can run on your own platform. Sources: Microsoft Learn, checked on 2026-09-30. Example organisation: Finance owns the data (core data domain), Sales builds reports on it (consumer domain).

Your architecture checklist

#	Decide first	Answer
1	Where does it live?	Workspace per domain and stage
2	How do others read it?	Shortcut to Gold, never a copy
3	What competes for compute?	Capacity per workload
4	How does it reach Prod?	Git, then fabric-cicd to Prod
5	Who says yes to access?	Owner per domain, roles in OneLake

1 Where does it live? Workspace per domain and stage

Month six: “Where is the current version?”

- **Decision.** Core data domains get one workspace per medallion layer and stage: Finance-Dev-Bronze, Finance-Dev-Silver, Finance-Dev-Gold, and the same for Test and Prod. Consumer domains get one workspace per stage, organised with folders per team: Sales-Dev-Reports, Sales-Test-Reports, Sales-Prod-Reports. Every workspace, Dev and Test included, sits in its Fabric domain. Names follow {Domain} - {Env} - {Purpose}, agreed before the first workspace.
- **Why it works in Fabric.** The workspace is the unit for roles, Git and deployment pipelines. Microsoft recommends one workspace per lakehouse layer: “we recommend that you create each lakehouse in its own, separate workspace”, for “more control and better governance at the layer level”.
- **Pitfall.** Expecting the domain or a folder to control access. Domain assignment “doesn’t affect item visibility or accessibility”, and folders inherit the workspace’s permissions. Access comes from workspace roles (answer 5).
- **Links.** [Deployment patterns](#) · [Medallion architecture](#) · [Fabric domains](#) · [Workspace naming](#) · repo: naming.md, ADR-0001

Monday check: A new colleague finds Sales Prod in 30 seconds. Every workspace name matches the pattern; no usage appears under “No domain” in the Chargeback app.

2 How do others read it? Shortcut to Gold, never a copy

Month six: “Which revenue number is right?”

- **Decision.** The core data domain that owns the numbers runs Bronze, Silver and Gold. Every consumer, Finance’s own reports included, reads Gold through a shortcut in its own workspace. A copy needs a written reason.
- **Why it works in Fabric.** A shortcut points to the data instead of copying it, so a change in Gold is visible to every reader at once. Microsoft’s data mesh pattern: “Have consumer domains use shortcuts to reference producer data products rather than copying them”. The reader’s capacity pays for its reads.
- **Pitfall.** A copy “just for performance”, which becomes the second revenue number. Microsoft’s own rule: “Record the reason whenever you create a synchronized copy”. Second pitfall: readers need access at both ends; OneLake shortcuts pass the reader’s identity to Gold by default.
- **Links.** [OneLake patterns](#) · [Shortcuts in a lakehouse](#) · [OneLake shortcut security](#) · [OneLake consumption](#)

Monday check: Count the tables named like a Gold table that live in another lakehouse and are not shortcuts. The target is zero.

3 What competes for compute? Capacity per workload

Month six: "Why is it slow on Monday?"

- **Decision.** Separate capacities for scheduled jobs (the Prod layer workspaces), for the reports people open (the Prod consumer workspaces), and for Dev and Test (paused outside working hours). Capacity names are lowercase letters and digits and can never change, for example `fcprodjobseu01`.
- **Why it works in Fabric.** Throttling is per capacity: when one is overloaded, the others keep running. For shortcut reads across capacities, "the throttling state of the consuming capacity determines whether it throttles calls to the item". On an overloaded capacity, Fabric delays interactive requests first, then rejects them.
- **Pitfall.** Buying a bigger SKU before looking. Microsoft: "Only sometimes is slow performance due to capacity throttling". Second pitfall: capacity overage is on by default for new capacities, billed at three times the pay-as-you-go rate, with "No performance boost"; decide the threshold on purpose.
- **Links.** [Throttling policy](#) · [Capacity planning part 3](#) · [Surge protection](#) · [Capacity overage](#) · repo: ADR-0003

Monday check: Open the Capacity Metrics app for Monday 08:00 to 10:00: does the reports capacity show interactive delay or rejection? If yes, which workspaces caused the load?

4 How does it reach Prod? Git, then fabric-cicd to Prod

Month six: "Who changed Prod?"

- **Decision.** Dev workspaces are connected to Git, one folder per workspace; changes arrive by pull request. A release pipeline runs fabric-cicd to Test and, after approval, to Prod, under a service principal. No builder or Contributor role for humans in Test and Prod. The admins group (two named people, ideally just-in-time via PIM) is break-glass only. Every human change shows up in the audit log.
- **Why it works in Fabric.** Git integration works per workspace, so each stage is its own workspace. Microsoft on fabric-cicd: "the Fabric product team officially supports and recommends it as a best practice". Data never travels, configuration does: a variable library value set per stage, or one parameter file with a value per stage, swaps IDs, schedules live in the item definition, and a post-deploy step runs the schema notebook, the load and the refresh.
- **Pitfall.** Stage names in item names (`finance_gold_dev`): every reference becomes a parameter. Keep item names identical in every stage. Second pitfall: expecting data or role memberships to arrive with the deployment; the lakehouse arrives empty, and Entra members of OneLake roles are ignored by lakehouse CI/CD.
- **Also know.** Reports and semantic models are still in preview in Git integration. fabric-cicd deploys the whole folder every time, not a diff (fabric-cicd docs). Same names cover references by lakehouse name; references across workspaces (Silver reading Bronze, a shortcut to Gold) need an item reference in the variable library or a parameter file rule.
- **Links.** [Fabric CI/CD best practices](#) · [Git integration](#) · [Deployment process](#) · [Audit log](#) · repo: ADR-0004

Monday check: When did a person, not the service principal, last change an item in a Prod workspace? The Microsoft Purview audit log answers it.

5 Who says yes to access? Owner per domain, roles in OneLake

Month six: "Who can see this report?"

- **Decision.** Each domain has an owner from the business who approves the members of its groups: admins, builders, readers. Groups, not people, get workspace roles. Readers get Viewer on the Gold workspace plus a OneLake security role on Gold with its tables, rows and columns, defined once at the data.
- **Why it works in Fabric.** Access comes from workspace roles and OneLake roles, not from the domain. OneLake security (generally available since May 2026) is enforced by Spark, the SQL analytics endpoint in user's identity mode and Direct Lake on OneLake. A Viewer sees no data until a OneLake role grants it.
- **Pitfall.** Three ways to lose it: readers made Contributor "so the SQL endpoint works" (Admin, Member and Contributor bypass OneLake roles); readers left in the DefaultReader role ("Otherwise, they keep full access to the data"); the SQL analytics endpoint still in delegated mode, which is the default for new endpoints.
- **Also know.** Consumer lakehouses with shortcuts to Gold need user's identity mode too: in delegated mode, shortcuts to tables with row or column rules aren't accessible through the SQL analytics endpoint. Build report models as Direct Lake on OneLake; Direct Lake on SQL reads shortcuts with the item owner's identity and, once the endpoint uses the reader's identity, falls back to DirectQuery 100% of the time. Row-level security is static: no dynamic or multi-table rules. Renaming a Gold table breaks its roles.
- **Links.** [Data access control model](#) · [Best practices for OneLake security](#) · [Table, column and row security](#) · [SQL analytics endpoint](#) · [Direct Lake security](#) · repo: ADR-0005

Monday check: For every Gold and every consumer lakehouse with shortcuts to Gold: endpoint in user's identity mode? DefaultReader emptied or edited? Every role member a security group?

Then the technology: Lakehouse all the way

- **Decision.** Lakehouse, Lakehouse, Lakehouse for Bronze, Silver and Gold. A Warehouse where Gold needs T-SQL writes, stored procedures or multi-table transactions.
- **Why it works in Fabric.** Change data feed on Delta tables lets the next layer read only the rows that changed; materialized lake views use it for incremental refresh. Shortcuts land in a lakehouse. One permission model: a Gold Warehouse adds T-SQL security that applies inside the warehouse only; through a shortcut, users "might see the full warehouse data". Lakehouse and Warehouse share Delta in OneLake and the same SQL engine, so a Warehouse can be added later without moving the lake.
- **Pitfall.** Switching change data feed on late: it doesn't backfill. Set `delta.enableChangeDataFeed = true` when you create Silver and Gold tables. Materialized lake views need schema-enabled lakehouses and change data feed on every source for incremental refresh.
- **Links.** [Change data feed](#) · [Optimal refresh for materialized lake views](#) · [Warehouse or Lakehouse](#) · repo: ADR-0002

Monday check: Do your Silver and Gold tables have change data feed on? Do materialized lake views refresh incrementally, or fully every time?

And keep the why: an ADR per decision

- **Decision.** One architecture decision record per decision: context, decision, options considered and why they lost, trade-offs, confidence, status, date, owner. Append-only: a changed decision gets a new record that supersedes the old one. Stored in Git.
- **Why.** Microsoft Well-Architected: record the options you ruled out and the confidence of each decision; don't edit accepted records.
- **Pitfall.** A record that only says what, never why; after a year nobody can tell whether it still applies.
- **Links.** [Maintain an ADR](#) · repo: template and five example ADRs

Monday check: Is there an ADR, with an owner and a date, for each of your five answers?

Everything from the session (slides, this cheatsheet, ADR template and five examples, naming reference, links per answer): <https://github.com/TheTrustedAdvisor/fabcon-europe-2026> · **Send me your month-six sentence:** <https://www.linkedin.com/in/matthias-falland/>